

- AWS Integration Architect — L1 Interview Q&A (Part 2)
 - Section 5: Data Integration & ETL (Q25–Q28)
 - Q25. Design a real-time inventory sync between OMS, E-commerce, and ERP.
 - Q26. Compare S3 vs DynamoDB vs RDS for integration data storage.
 - Q27. How do you implement Change Data Capture (CDC) for real-time data sync?
 - Q28. How do you handle large file processing (e.g., daily product catalog from ERP)?
 - Section 6: DevOps & IaC (Q29–Q32)
 - Q29. How do you structure Terraform for integration workloads?
 - Q30. How do you implement CI/CD for Lambda-based integrations?
 - Q31. How do you handle environment-specific configuration across dev/staging/prod?
 - Q32. What IaC best practices do you enforce for integration teams?
 - Section 7: Scenario-Based & Behavioral (Q33–Q37)
 - Q33. An integration is dropping messages during Black Friday traffic spikes. How do you fix it?
 - Q34. Your Lambda-based integration has cold start latency of 5 seconds. How do you reduce it?
 - Q35. You need to migrate a legacy ESB (MuleSoft/TIBCO) to AWS. What's your approach?
 - Q36. How do you estimate cost for a serverless integration architecture?
 - Q37. Tell us about a complex integration challenge you solved.
 - Quick Reference — Key Numbers to Know

AWS Integration Architect — L1 Interview Q&A (Part 2)

Role: AWS Integration Architect | **Level:** 10-15 Years | **Duration:** 1 Hour L1

Section 5: Data Integration & ETL (Q25–Q28)

Q25. Design a real-time inventory sync between OMS, E-commerce, and ERP.

Answer:

```
OMS (inventory change) → EventBridge ("inventory.updated")
  → Rule 1: Lambda → E-commerce platform API (update stock)
  → Rule 2: SQS → Lambda → ERP (batch sync every 5 min)
  → Rule 3: Lambda → DynamoDB (inventory cache for low-latency reads)

E-commerce (sale) → API Gateway → Lambda → EventBridge ("stock.reserved")
  → OMS update
  → If stock < threshold → SNS → procurement alert
```

Why EventBridge as hub: Single source of truth for inventory events. Any new consumer (analytics, ML forecasting) subscribes without touching producers.

Consistency handling: Eventual consistency with DynamoDB as read cache. Optimistic locking on inventory updates (**version** attribute) prevents overselling.

Q26. Compare S3 vs DynamoDB vs RDS for integration data storage.

Answer:

Criteria	S3	DynamoDB	RDS/Aurora
Best for	Files, data lake, ETL staging	Key-value, high-throughput, low-latency	Relational, complex queries, joins
Scale	Unlimited	Auto-scales	Vertical (read replicas help)
Latency	50-100ms	1-5ms	5-20ms
Cost	\$0.023/GB/month	Pay per request or provisioned	Instance-based (\$\$\$)

Criteria	S3	DynamoDB	RDS/Aurora
Integration use	ETL staging, file drops, archives	API cache, session store, idempotency keys	Transaction data, reporting

My rule of thumb:

- **Hot data** (API responses, cache): DynamoDB
- **Warm data** (recent transactions, operational): Aurora
- **Cold data** (archives, analytics): S3 + Athena

Q27. How do you implement Change Data Capture (CDC) for real-time data sync?

Answer:

Pattern 1: DynamoDB Streams → Lambda

DynamoDB table change → DynamoDB Stream → Lambda → EventBridge

Every insert/update/delete triggers Lambda. Great for event sourcing.

Pattern 2: RDS/Aurora → DMS CDC → Kinesis/S3

Aurora (binlog) → DMS replication task (CDC) → Kinesis Data Streams → Lambda

Real-time replication of relational DB changes without app code changes.

Pattern 3: Debezium on MSK (Kafka)

Source DB → Debezium connector → Amazon MSK → Lambda/consumer

For complex CDC with schema evolution and exactly-once semantics.

Key consideration: CDC captures ALL changes (including internal system updates).

Add filtering in Lambda to process only business-relevant events.

Q28. How do you handle large file processing (e.g., daily product catalog from ERP)?

Answer:

```
ERP → SFTP (AWS Transfer Family) → S3 (raw/)
  → S3 Event → Lambda (validate + split into chunks)
  → S3 (chunks/) → SQS (one message per chunk)
  → Lambda (process chunk, upsert to DynamoDB)
  → S3 (processed/) → Glue (analytics load to Redshift)
```

Why chunk? A 500MB catalog file with 1M products can't be processed by a single Lambda (15-min timeout, 10GB memory). Split into 1000-record chunks → 1000 parallel Lambda executions → processed in seconds.

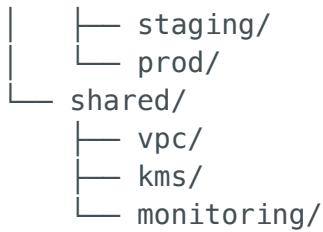
Error handling: Each chunk is independent. Failed chunks go to DLQ, successful ones are committed. Redrive only the failed chunks after fixing.

Section 6: DevOps & IaC (Q29–Q32)

Q29. How do you structure Terraform for integration workloads?

Answer:

```
terraform/
├── modules/
│   ├── api-gateway/      # Reusable API Gateway module
│   ├── lambda-function/  # Standard Lambda with DLQ, alarms, IAM
│   ├── sqs-queue/        # SQS + DLQ + CloudWatch alarms
│   ├── eventbridge/      # Event bus + rules + targets
│   └── step-function/    # State machine + IAM + logging
├── environments/
│   └── dev/
```



Key practices:

- **Modules** for every integration pattern (reuse across projects)
- **Remote state** in S3 + DynamoDB locking
- **Workspaces or separate state files** per environment
- **terraform plan in CI** (PR comment with plan output)
- **terraform apply only on main merge** with approval gate
- **Drift detection** via scheduled **terraform plan** (CloudWatch Events → CodeBuild)

Q30. How do you implement CI/CD for Lambda-based integrations?

Answer:

```
# CI Pipeline
1. Lint + Unit Test (pytest/jest)
2. SAST (SonarCloud)
3. SCA (safety/npm audit for dependency CVEs)
4. Package Lambda (zip or container)
5. Terraform plan (show infra changes)

# CD Pipeline
6. Deploy to dev (terraform apply)
7. Integration tests (hit real endpoints)
8. Deploy to staging (terraform apply)
9. Smoke tests
10. Manual approval gate
11. Deploy to prod (terraform apply)
12. Canary monitoring (5 min)
```

Lambda deployment strategies:

- **All-at-once:** Default. Simple but risky.

- **Canary (CodeDeploy):** Route 10% traffic to new version, monitor errors for 5 min, then shift 100%.
- **Linear (CodeDeploy):** Shift 10% every 2 minutes over 20 minutes.
- **Alias + Weighted:** API Gateway routes to Lambda alias; update alias to point to new version.

Q31. How do you handle environment-specific configuration across dev/staging/prod?

Answer:

Config Type	Storage	Example
Infrastructure	Terraform tfvars per env	VPC CIDR, instance sizes
App config	SSM Parameter Store	API endpoints, feature flags
Secrets	Secrets Manager	DB passwords, API keys
Lambda env vars	Terraform-managed	ENVIRONMENT=prod , LOG_LEVEL=INFO

Pattern: Lambda reads config from SSM at cold start and caches it:

```
# Cached outside handler – shared across invocations
import boto3
ssm = boto3.client('ssm')
config = ssm.get_parameter(Name=f'/{env}/order-service/erp-endpoint')
```

Never: Different code per environment. Same artifact, different config.

Q32. What IaC best practices do you enforce for integration teams?

Answer:

1. **No ClickOps** — Every resource in Terraform. No manual Console changes.

2. **Module-first** — Common patterns (Lambda+SQS+DLQ) packaged as modules
 3. **PR-based workflow** — Plan on PR, apply on merge. No direct apply.
 4. **State locking** — S3 backend + DynamoDB lock table
 5. **Tagging** — Every resource: **Project**, **Environment**, **Owner**, **CostCenter**
 6. **Security scanning** — Checkov/tfsec in CI before apply
 7. **Least privilege** — IAM policies scoped to specific resources
 8. **Outputs** — Every module outputs ARNs/URLs for cross-module references
-

Section 7: Scenario-Based & Behavioral (Q33–Q37)

Q33. An integration is dropping messages during Black Friday traffic spikes. How do you fix it?

Answer:

Immediate (within 1 hour):

1. Check SQS DLQ — are messages failing or being throttled?
2. Check Lambda concurrent executions — hitting account limit (1000)?
3. Increase Lambda reserved concurrency for critical functions
4. If API Gateway: increase throttle limits via usage plan

Short-term (within 1 day):

1. Add SQS between API Gateway and Lambda (buffer the spike)
2. Increase Lambda batch size and batch window (process more per invocation)
3. Switch from synchronous to asynchronous processing where possible
4. Enable Lambda provisioned concurrency for critical paths (eliminates cold starts)

Long-term:

1. Implement auto-scaling DynamoDB (on-demand mode)
2. Design for 10x normal traffic capacity
3. Load test before every peak season

Q34. Your Lambda-based integration has cold start latency of 5 seconds. How do you reduce it?

Answer:

Root cause analysis:

- Java/Spring Boot Lambda = 5-10s cold start (JVM + framework init)
- VPC-attached Lambda adds 1-2s (ENI creation — largely fixed now with Hyperplane)

Solutions (in priority order):

1. **Provisioned Concurrency** — Pre-warm N instances. Eliminates cold start completely. Cost: pay for idle capacity.
 2. **SnapStart (Java only)** — Takes a snapshot of initialized JVM. Cold start drops from 5s to <1s. Free. Enable with `SnapStart: {ApplyOn: PublishedVersions}`.
 3. **Reduce package size** — Use Lambda layers for dependencies. Smaller zip = faster load.
 4. **Switch runtime** — Python/Node.js cold start is 100-300ms vs Java's 3-10s. Consider for latency-sensitive APIs.
 5. **Keep-alive pings** — CloudWatch Events pings Lambda every 5 min. Hacky but works for low-traffic functions.
 6. **Move to containers** — For complex Java apps, ECS Fargate with ALB may be better than Lambda.
-

Q35. You need to migrate a legacy ESB (MuleSoft/TIBCO) to AWS. What's your approach?

Answer:

Assessment (2 weeks):

- Inventory all ESB flows (100+ typical for retail)
- Classify: API orchestration, file transfer, event routing, data transformation
- Map each flow to an AWS service

Migration mapping:

ESB Capability	AWS Replacement
API orchestration	API Gateway + Step Functions
Message routing	EventBridge + SQS/SNS
Data transformation	Lambda (simple) or Glue (complex ETL)
File transfer	S3 + AWS Transfer Family
Scheduling	EventBridge Scheduler
Error handling	DLQ + CloudWatch Alarms
Monitoring	X-Ray + CloudWatch

Strangler fig pattern: Migrate one flow at a time. Route traffic from ESB to AWS for migrated flows while legacy flows continue on ESB. Gradually strangle the ESB until it's decommissioned.

Q36. How do you estimate cost for a serverless integration architecture?

Answer:

Key cost drivers:

Service	Pricing Model	Typical Cost
Lambda	Per request + duration (GB-s)	1M requests/month = ~\$20
API Gateway	Per request (REST: 3.50/M, HTTP : 1.00/M)	10M req = \$10-35
SQS	\$0.40 per 1M requests	10M msg = \$4

Service	Pricing Model	Typical Cost
EventBridge	\$1.00 per 1M events	5M events = \$5
Step Functions	\$25 per 1M state transitions	100K workflows = \$2.50
DynamoDB	On-demand: 1.25/1M writes, 0.25/1M reads	Varies
S3	\$0.023/GB storage + request costs	Low

Optimization levers:

- Lambda: Right-size memory (use AWS Lambda Power Tuning tool)
- SQS: Use batching (10 messages = 1 request = 1/10th cost)
- DynamoDB: Reserved capacity for predictable workloads (up to 77% savings)
- API Gateway: Use HTTP API instead of REST API (70% cheaper)

Q37. Tell us about a complex integration challenge you solved.

Answer (STAR format):

Situation: Retail client with 200+ stores needed real-time inventory visibility across POS, E-commerce, warehouse, and ERP. Legacy batch integration (nightly file drops) caused overselling.

Task: Design and implement near-real-time inventory sync across all systems with <30s latency.

Action:

1. Designed event-driven architecture: POS/WMS → EventBridge → fan-out to all consumers
2. DynamoDB as inventory cache (single-digit ms reads for e-commerce)
3. Step Functions saga for stock reservation (reserve → confirm → release on timeout)
4. SQS FIFO for ordered processing per SKU (prevents race conditions)
5. Implemented idempotency to handle duplicate events from POS systems
6. Glue ETL for nightly reconciliation between real-time cache and ERP source-of-truth

Result: Inventory sync latency reduced from 24 hours to <10 seconds. Overselling incidents reduced by 95%. System handled 50K events/minute during peak without issues.

Quick Reference — Key Numbers to Know

Metric	Value
Lambda max timeout	15 minutes
Lambda max memory	10,240 MB
Lambda max package size	50 MB (zip), 250 MB (unzipped), 10 GB (container)
API Gateway max timeout	29 seconds
SQS max message size	256 KB (use S3 for larger)
SQS max retention	14 days
SQS FIFO throughput	300 msg/s (3000 with batching)
EventBridge max event size	256 KB
Step Functions max execution	1 year (Standard), 5 min (Express)
DynamoDB max item size	400 KB
Glue max DPU	100 (default)

Good luck with the interview tomorrow, Pushparaj! 🚀